PROJECT REPORT

ON

RAILWAY RESERVATION SYSTEM USING QUEUE IN C

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Subject: Data Structures and Algorithms

Submitted To:

Ms. Konika Rani

Assistant Professor

University Institute of Computing

Submitted By:

Navneet Kaur

UID/Roll No: 25BCA10043

Session: 2025-26

CHANDIGARH UNIVERSITY

1. ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my teacher, **Ms. Konila Rani**, for her invaluable guidance and supervision throughout the development of this project. Her insights into Data Structures and Algorithms have been instrumental in helping me understand the practical implementation of Queues.

I am also thankful to **Chandigarh University** for providing me with the resources and environment necessary to complete this academic endeavor.

Lastly, I would like to thank my parents and friends for their constant support and encouragement during the preparation of this report.

Navneet Kaur
UID: 25BCA10043

2. ABSTRACT

The "Railway Reservation System" is a software application designed to manage the process of booking and cancelling train tickets. In a real-world scenario, passenger demand often exceeds the number of available seats. To handle this efficiently and fairly, the **Queue Data Structure** is employed.

This project implements a simplified version of a reservation system using the C programming language. It utilizes the **First-In-First-Out (FIFO)** principle of queues to manage a "Waiting List." When the confirmed seats are full, subsequent passengers are added to a waiting queue. If a confirmed passenger cancels their ticket, the first person in the waiting queue is automatically moved to the confirmed list. This report covers the theoretical background, algorithmic design, source code, and system analysis of the project.

3. TABLE OF CONTENTS

1. Introduction ..... Page 4

2. Problem Statement ..... Page 5

3. Objectives of the Project ..... Page 5

4. Scope of the Project ..... Page 6

5. Data Structure Used: Queue ..... Page 7

6. System Requirements ..... Page 8

7. Methodology ..... Page 9

8. System Design ..... Page 10

9. Algorithm ..... Page 11

10. Source Code ..... Page 12

11. Advantages & Limitations ..... Page 14

12. Future Scope & Conclusion ..... Page 15

13. Bibliography ..... Page 15

Introduction

4.1 Railway Reservation Systems

A Railway Reservation System (RRS) is a critical infrastructure for any nation's transport system. It allows passengers to check train availability, book seats, and cancel journeys. The complexity of these systems lies in managing huge volumes of data and ensuring that the seat allocation process is transparent and logical. In modern times, the digitalization of this process has reduced physical queues and manual errors significantly.

4.2 Importance of Data Structures

Data structures are not just theoretical concepts; they are the backbone of efficient software. Choosing the right data structure can reduce the time complexity of an operation from hours to

4.3 Brief Explanation of Queue

A **Queue** is a linear data structure that follows the **FIFO (First-In-First-Out)** principle. Imagine a line of people waiting at a ticket counter: the person who arrives first is the first to be served. In programming, a Queue has two main ends: Rear (where elements are added) and Front (where elements are removed).

milliseconds. In a reservation system, we need to manage lists of passengers, find specific seats, and maintain an order of priority. Without structured data management, the system would face issues like "double-booking" or unfair seat allocation.

5. PROBLEM STATEMENT

Manual railway booking is prone to errors, slow, and difficult to manage when dealing with waiting lists. The primary challenge is:

- How to store passenger data efficiently.
- How to manage a "Waiting List" such that the person who booked first gets the first available seat upon cancellation.
- How to automate the transition from "Waiting" status to "Confirmed" status.

This project addresses these issues by creating a digital system where logic replaces manual record-keeping.

6. OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To implement a functional Railway Reservation System in C.
- To demonstrate the practical application of the **Queue Data Structure**.
- To provide a user-friendly interface for booking and cancellation.
- To ensure data integrity by using structures and arrays to store passenger details.
- To understand how the FIFO principle applies to real-world waiting list management.

7. SCOPE OF THE PROJECT

While commercial systems like IRCTC are massive, this project focuses on the core logic. The scope includes:

- Booking tickets up to a specific limit (e.g., 3 confirmed seats for demonstration).
- Adding passengers to a Waiting List once seats are full.
- Automatic promotion of waiting list passengers upon cancellation.
- Displaying the current status of all passengers.
- Basic data entry (Name, Age, and Ticket ID).

8. DATA STRUCTURE USED: QUEUE

A Queue is a collection of elements where removal happens at one end and addition at another. In this project, we use a **Linear Queue**.

8.1 FIFO Concept

First-In-First-Out ensures fairness. In railway systems, if a Seat becomes vacant, it shouldn't go to the person who just arrived; it must go to the person who has been waiting the longest. This is exactly what the Queue provides.

8.2 Real-Life Example

Consider a train with only 10 seats. Passenger 1 to 10 get "Confirmed" status. Passenger 11 arrives; they are placed in the **Waiting Queue** (Position 1). Passenger 12 arrives; they are placed in the **Waiting Queue** (Position 2). If Passenger 3 cancels, the system looks at the **Front** of the Waiting Queue. Passenger 11 is "Dequeued" from the waiting list and moved to the "Confirmed" list.

9. SYSTEM REQUIREMENTS

Category	Requirement
Processor	Intel i3 or higher
RAM	Minimum 4GB
Operating System	Windows 10/11 or Linux
Language	C (C99 standard)
Compiler	GCC / Code::Blocks

10. METHODOLOGY

The project was developed using a modular approach:

1. **Requirement Analysis:** Identifying the need for confirmed and waiting lists.
2. **Data Design:** Creating a 'struct' in C to hold passenger data (ID, Name, Age).
3. **Logic Development:** Writing functions for enqueue (adding to waiting) and dequeue (moving to confirmed).
4. **Coding:** Implementing the main menu and switch-case logic.
5. **Testing:** Running the code with various scenarios.

11. SYSTEM DESIGN

The system uses two primary data storage areas:

- **Confirmed Array:** A fixed-size array that stores passengers who have a seat.
- **Waiting Queue:** A queue structure that holds passengers who will get a seat only if someone in the Confirmed Array cancels.

Logic Flow: If (count < MAX) then Add to Confirmed Array. Else, Add to Waiting Queue. On Cancellation, remove from Confirmed and if Queue is not empty, move Front of Queue to the empty spot in Confirmed.

12. ALGORITHM

Booking Algorithm

1. START
2. Check if count < MAX_SEATS.
3. If YES, input details and set status to CONFIRMED.
4. If NO, check if queue_count < MAX_WAITING.
5. If YES, Enqueue to Waiting Queue with status WAITING.
6. Else, Print "No tickets available".
7. END

13. SOURCE CODE

```
#include    <stdio.h>
#include   <string.h>
#include <stdlib.h>

#define MAX 3 // Maximum Confirmed Seats
#define Q_MAX 2 // Maximum Waiting List Capacity

struct Passenger {
      int id;
      char name[50];
      int age;
};

struct                        Passenger
confirmed[MAX];                struct
Passenger    waiting[Q_MAX];    int
confirmed_count = 0;
int wait_front = -1;
int wait_rear = -1;
int id_counter = 101;
void enqueue(struct Passenger p) {
      if (wait_rear == Q_MAX - 1) {
            printf("\nWaiting List full!\n");
      } else {
            if (wait_front == -1) wait_front = 0;
            wait_rear++;
            waiting[wait_rear] = p;
            printf("\nAdded to Waiting List.\n");
      }
}

struct Passenger dequeue() {
      struct Passenger p = waiting[wait_front];
      if (wait_front == wait_rear) {
            wait_front = wait_rear = -1;
      } else {
            wait_front++;
      }
      return p;
}

void book_ticket() {
      struct Passenger p;
```

```
    p.id = id_counter++;
    printf("Enter Name: "); scanf("%s", p.name);
    printf("Enter Age: "); scanf("%d", &p.age);

    if (confirmed_count < MAX) {
        confirmed[confirmed_count++] = p;
        printf("\nConfirmed! ID: %d\n", p.id);
    } else {
        enqueue(p);
    }
}

void cancel_ticket() {
int id, found = 0;
printf("Enter ID to cancel: "); scanf("%d", &id);
for (int i = 0; i < confirmed_count; i++) {
        if (confirmed[i].id == id) {
            found = 1;
            if (wait_front != -1) {
                confirmed[i] = dequeue();
                printf("Waiting list passenger moved to Confirmed.\n");
            } else {
                for (int j = i; j < confirmed_count - 1; j++)
                    confirmed[j] = confirmed[j+1];
                confirmed_count--;
            }
            break;
        }
    }
    if (!found) printf("ID not found.\n");
}

void display() {
    printf("\n--- STATUS ---\n");
    for (int i = 0; i < confirmed_count; i++)
        printf("CONFIRMED - ID: %d, Name: %s\n", confirmed[i].id,
confirmed[i].name);
    if(wait_front != -1)
        for(int i = wait_front; i <= wait_rear; i++)
            printf("WAITING - ID: %d, Name: %s\n", waiting[i].id, waiting[i].name);
}

int main() {
    int choice;
    while(1) {
        printf("\n1.Book 2.Cancel 3.Display 4.Exit: ");
        scanf("%d", &choice);
        if(choice==1) book_ticket();
        else if(choice==2) cancel_ticket();
        else if(choice==3) display();
```

```
            else exit(0);
        }
        return 0;
    }
```

14. ADVANTAGES & LIMITATIONS

Advantages

- **Efficiency:** Constant time complexity for Queue operations.
- **Fairness:** Strict FIFO order.
- **Lightweight:** Uses minimal system resources.

Limitations

- **Static Memory:** Array size is fixed.
- **Volatile:** Data is lost on exit (no database/file storage).

15. CONCLUSION

The Railway Reservation System project successfully demonstrates the utility of the **Queue Data Structure** in managing real-world scenarios. By applying the FIFO principle, the system ensures a systematic and error-free approach to ticket booking and cancellation. This project provides a solid foundation for understanding resource management.

16. BIBLIOGRAPHY

1. Lipschutz, S. (2017). *Data Structures with C*. McGraw Hill.
2. Kanetkar, Y. (2020). *Let Us C*. BPB Publications.
3. GeeksforGeeks: Queue Data Structure Documentation.